



PROGRAM STUDI TEKNIK KOMPUTER
JURUSAN INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS JENDERAL SOEDIRMAN

MODUL PRAKTIKUM PEMROGRAMAN WEB II

Framework Web Modern, RESTful API, dan Integrasi Backend Frontend

Laravel 13 (PHP) dan Fiber v3 (Go)

MODUL 3

Basis Data, Migration, dan Eloquent ORM

Disusun oleh

Tim Dosen Pengampu Matakuliah Pemrograman Web II

**PURWOKERTO
TAHUN AKADEMIK 2026/2027**

MODUL 3

Basis Data, Migration, dan Eloquent ORM

Praktikum Pemrograman Web II | Teknik Komputer UNSOED

A. Tujuan Praktikum

Setelah menyelesaikan modul ini mahasiswa mampu:

1. Mengonfigurasi koneksi basis data pada aplikasi Laravel.
2. Merancang dan menjalankan migration untuk membentuk struktur tabel.
3. Membuat model Eloquent dan melakukan operasi CRUD.
4. Menerapkan relasi antar tabel pada level model.
5. Mengisi data awal menggunakan seeder dan factory.

B. Alat dan Bahan

Proyek Laravel 13 hasil Modul 2, server MySQL atau MariaDB, dan klien basis data seperti DBeaver atau phpMyAdmin.

C. Dasar Teori

C.1 ORM

Object Relational Mapping memetakan baris tabel basis data menjadi objek pada bahasa pemrograman. Dengan ORM, pengembang menulis kode berorientasi objek alih-alih menulis kueri SQL secara manual.

Keuntungan ORM meliputi keterbacaan kode, kemudahan perpindahan sistem basis data, dan perlindungan bawaan terhadap SQL injection. Kekurangannya, kueri yang dihasilkan tidak selalu optimal sehingga tetap perlu pemahaman SQL.

C.2 Migration

Migration adalah berkas PHP yang mendeskripsikan perubahan struktur basis data. Berkas ini disimpan bersama kode sehingga seluruh anggota tim memiliki struktur tabel yang identik. Migration menghilangkan kebiasaan bertukar berkas dump SQL secara manual.

C.3 Jenis Relasi

Relasi	Contoh
One to One	Mahasiswa memiliki satu kartu tanda mahasiswa
One to Many	Program studi memiliki banyak mahasiswa

Relasi
Many to Many

Contoh
Mahasiswa mengambil banyak
matakuliah

C.4 Seeder dan Factory

Seeder mengisi tabel dengan data awal. Factory menghasilkan data acak yang realistis untuk keperluan pengujian.

D. Langkah Praktikum

Langkah 1: Membuat Basis Data

Buat basis data baru melalui klien MySQL.

```
CREATE DATABASE pemweb2_praktikum CHARACTER SET utf8mb4 COLLATE  
utf8mb4_unicode_ci;
```

Langkah 2: Mengonfigurasi Koneksi

Buka berkas .env pada proyek Laravel, lalu sesuaikan bagian berikut.

```
DB_CONNECTION=mysql  
DB_HOST=127.0.0.1  
DB_PORT=3306  
DB_DATABASE=pemweb2_praktikum  
DB_USERNAME=root  
DB_PASSWORD=
```

Bersihkan cache konfigurasi.

```
php artisan config:clear
```

Uji koneksi.

```
php artisan db:show
```

Langkah 3: Membuat Migration Program Studi

```
php artisan make:migration create_program_studis_table
```

Buka berkas migration yang baru dibuat pada folder database/migrations, lalu isi method up.

```
public function up(): void  
{  
    Schema::create('program_studis', function (Blueprint $table) {  
        $table->id();  
        $table->string('kode', 10)->unique();  
        $table->string('nama', 100);  
        $table->string('jenjang', 10);  
        $table->timestamps();  
    });  
}
```

```
    });  
}
```

Langkah 4: Membuat Migration Mahasiswa

```
php artisan make:migration create_mahasiswas_table
```

Isi method up sebagai berikut.

```
public function up(): void  
{  
    Schema::create('mahasiswas', function (Blueprint $table) {  
        $table->id();  
  
$table->foreignId('program_studi_id')->constrained('program_studis')->cascade  
OnDelete();  
        $table->string('nim', 20)->unique();  
        $table->string('nama', 100);  
        $table->string('email', 100)->unique();  
        $table->year('angkatan');  
        $table->decimal('ipk', 4, 2)->default(0);  
        $table->boolean('aktif')->default(true);  
        $table->timestamps();  
    });  
}
```

Langkah 5: Menjalankan Migration

```
php artisan migrate
```

Periksa tabel yang terbentuk pada klien basis data. Untuk melihat status migration gunakan perintah berikut.

```
php artisan migrate:status
```

Langkah 6: Membuat Model

```
php artisan make:model ProgramStudi
```

```
php artisan make:model Mahasiswa
```

Isi berkas app/Models/ProgramStudi.php.

```
<?php
```

```
namespace App\Models;
```

```
use Illuminate\Database\Eloquent\Model;  
use Illuminate\Database\Eloquent\Relations\HasMany;
```

```
class ProgramStudi extends Model  
{  
    protected $table = 'program_studis';
```

```

        protected $fillable = ['kode', 'nama', 'jenjang'];

        public function mahasiswa(): HasMany
        {
            return $this->hasMany(Mahasiswa::class);
        }
    }
}

```

Isi berkas app/Models/Mahasiswa.php.

```

<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;

class Mahasiswa extends Model
{
    protected $table = 'mahasiswas';

    protected $fillable = [
        'program_studi_id',
        'nim',
        'nama',
        'email',
        'angkatan',
        'ipk',
        'aktif',
    ];

    protected function casts(): array
    {
        return [
            'angkatan' => 'integer',
            'ipk' => 'decimal:2',
            'aktif' => 'boolean',
        ];
    }

    public function programStudi(): BelongsTo
    {
        return $this->belongsTo(ProgramStudi::class);
    }
}

```

Langkah 7: Membuat Factory Mahasiswa

php artisan make:factory MahasiswaFactory

Isi method definition pada database/factories/MahasiswaFactory.php.

```

public function definition(): array
{
    return [
        'program_studi_id' => 1,
        'nim' => 'H1A' . fake()->unique()->numberBetween(100000, 999999),
        'nama' => fake('id_ID')->name(),
        'email' => fake()->unique()->safeEmail(),
        'angkatan' => fake()->numberBetween(2021, 2025),
        'ipk' => fake()->randomFloat(2, 2.50, 4.00),
        'aktif' => fake()->boolean(85),
    ];
}

```

Langkah 8: Membuat Seeder

php artisan make:seeder ProgramStudiSeeder

Isi method run.

```

public function run(): void
{
    $daftar = [
        ['kode' => 'TK', 'nama' => 'Teknik Komputer', 'jenjang' => 'S1'],
        ['kode' => 'IF', 'nama' => 'Informatika', 'jenjang' => 'S1'],
        ['kode' => 'TE', 'nama' => 'Teknik Elektro', 'jenjang' => 'S1'],
    ];

    foreach ($daftar as $item) {
        ProgramStudi::create($item);
    }
}

```

Tambahkan baris import pada bagian atas berkas seeder.

```
use App\Models\ProgramStudi;
```

Panggil seeder tersebut pada database/seeder/DatabaseSeeder.php.

```

public function run(): void
{
    $this->call(ProgramStudiSeeder::class);

    Mahasiswa::factory()->count(30)->create();
}

```

Tambahkan import model pada berkas yang sama.

```
use App\Models\Mahasiswa;
```

Jalankan seeder.

```
php artisan migrate:fresh --seed
```

Langkah 9: Menguji Eloquent melalui Tinker

php artisan tinker

Jalankan perintah berikut satu per satu di dalam Tinker.

```
App\Models\Mahasiswa::count();
App\Models\Mahasiswa::first();
App\Models\Mahasiswa::where('angkatan', 2023)->get();
App\Models\Mahasiswa::orderBy('ipk', 'desc')->take(5)->pluck('nama', 'ipk');
App\Models\ProgramStudi::find(1)->mahasiswa()->count();
```

Keluar dari Tinker dengan mengetik exit.

Langkah 10: CRUD melalui Controller

Buat controller resource.

php artisan make:controller MahasiswaWebController --resource

Isi bagian penting pada app/Http/Controllers/MahasiswaWebController.php.

```
public function index()
{
    $daftarMahasiswa = Mahasiswa::with('programStudi')
        ->orderBy('nama')
        ->paginate(10);

    return view('mahasiswa.data', ['daftarMahasiswa' =>
$daftarMahasiswa]);
}

public function store(Request $request)
{
    $data = $request->validate([
        'program_studi_id' => ['required', 'exists:program_studis,id'],
        'nim' => ['required', 'string', 'max:20',
'unique:mahasiswas,nim'],
        'nama' => ['required', 'string', 'max:100'],
        'email' => ['required', 'email', 'unique:mahasiswas,email'],
        'angkatan' => ['required', 'integer', 'min:2000'],
    ]);

    Mahasiswa::create($data);

    return redirect()->route('mahasiswa.data')->with('sukses', 'Data
mahasiswa berhasil disimpan');
}
```

Tambahkan import berikut pada bagian atas berkas.

```
use App\Models\Mahasiswa;
```

Langkah 11: Menampilkan Data dengan Pagination

Buat berkas `resources/views/mahasiswa/data.blade.php`.

```
@extends('layouts.app')

@section('judul', 'Data Mahasiswa')

@section('konten')
    <h1 class="h3 mb-4">Data Mahasiswa</h1>

    @if (session('sukses'))
        <div class="alert alert-success">{{ session('sukses') }}</div>
    @endif

    <table class="table table-striped bg-white">
        <thead>
            <tr>
                <th>NIM</th>
                <th>Nama</th>
                <th>Program Studi</th>
                <th>Angkatan</th>
                <th>IPK</th>
            </tr>
        </thead>
        <tbody>
            @foreach ($daftarMahasiswa as $mahasiswa)
                <tr>
                    <td>{{ $mahasiswa->nim }}</td>
                    <td>{{ $mahasiswa->nama }}</td>
                    <td>{{ $mahasiswa->programStudi->nama }}</td>
                    <td>{{ $mahasiswa->angkatan }}</td>
                    <td>{{ $mahasiswa->ipk }}</td>
                </tr>
            @endforeach
        </tbody>
    </table>

    {{ $daftarMahasiswa->links() }}
@endsection
```

Daftarkan rute pada `routes/web.php`.

```
use App\Http\Controllers\MahasiswaWebController;
```

```
Route::get('/mahasiswa-data', [MahasiswaWebController::class,
'index'])->name('mahasiswa.data');
```

Langkah 12: Mengamati Kueri yang Dihasilkan

Tambahkan sementara kode berikut pada method index sebelum pengambilan data.

```
DB::listen(function ($kueri) {
    logger($kueri->sql);
});
```

Tambahkan import berikut.

```
use Illuminate\Support\Facades\DB;
```

Muat ulang halaman, lalu periksa berkas storage/logs/laravel.log. Amati perbedaan jumlah kueri ketika with('programStudi') dihapus. Peristiwa ini dikenal sebagai masalah N plus 1.

E. Tugas Praktikum

1. Tambahkan tabel matakuliahs dengan kolom kode, nama, sks, dan semester beserta model dan seedernya.
2. Buat relasi many to many antara mahasiswa dan matakuliahs melalui tabel pivot mahasiswa_matakuliah yang memiliki kolom tambahan nilai.
3. Tampilkan halaman detail mahasiswa yang memuat daftar matakuliah yang diambil beserta nilainya.
4. Buat query menggunakan Eloquent untuk menampilkan sepuluh mahasiswa dengan IPK tertinggi pada program studi Teknik Komputer.

F. Pertanyaan Pembahasan

1. Jelaskan fungsi properti \$fillable dan risiko yang muncul apabila properti tersebut diabaikan.
2. Apa perbedaan migrate:fresh, migrate:refresh, dan migrate:rollback?
3. Jelaskan masalah N plus 1 beserta cara mengatasinya berdasarkan pengamatan pada Langkah 12.

G. Rubrik Penilaian

Aspek	Bobot
Ketepatan rancangan migration dan relasi	30%
Implementasi model dan seeder	25%
Penyelesaian tugas praktikum	30%
Laporan dan pembahasan	15%